NeuroForge AI

User Manual

October 24, 2026

01 Your first training run	Installation, workspace, data and devices
02 From search space to model	Search, proxies, modules and Transformers
03 From model to deliverable	Copilot, model assets, SDK and figures
04 Keep your work moving	Logs, recovery, backups and upgrades

Available as HTML, PDF and Markdown, with runnable examples.

Contents

1. Meet NeuroForge AI 3
2. Installation and first launch 4
3. Desktop launcher and session control 5
4. Projects, data folders, and storage 6
5. Navigate the application 7
6. First tutorial: train a CNN 8
7. Dataset Studio 9
8. Architecture Studio 10
9. Visual Search Space Designer 11
10. Search Lab 12
11. Proxy analysis 13
12. Training Studio 14
13. Task states, pause, resume, and checkpoints 15
14. Visual Tensor Debugger and profiler 16
15. Module Library and reusable subgraphs 17
16. Transformer and Vision Transformer workflows 18
17. AI Architecture Copilot 19
18. Architecture Optimization Assistant 21
19. Model Ecosystem 22
20. Code → Visual and Architecture Diff 23
21. Custom Operator SDK 24
22. Experiment Lab and model export 25
23. Visual Data Pipeline 26
24. Export Studio, Figure Studio, and Neural CAD 27
25. Python SDK, CLI, and API 28
26. Troubleshooting 29
27. Backup, migration, upgrades, and removal 31
28. Three complete project recipes 32
29. Configuration and terminology reference 33
30. Working checklist, files, and further reading 35

Version 0.4.1 · October 24, 2026 · English edition

NeuroForge AI brings visual model design, architecture search, datasets, training, inspection, and architecture figures into one workspace. This guide follows the complete workflow: prepare data, design a model, explore candidates, train the selected architecture, and export the results.

Interface labels in this guide match the English application. Start with chapters 2–6 for your first run. Use chapters 7–16 to develop models and search spaces, chapters 17–25 for advanced workflows, and chapters 26–30 for maintenance and reference.

1. Meet NeuroForge AI

1.1 The workspace

The visual model graph is the center of a project. It holds operators, connections, input shapes, and module definitions. A project also connects the graph to a dataset version, a preprocessing pipeline, a search space, and experiment records.

Use Architecture Studio to create a model, Search Lab to explore alternatives, and Training Studio to fit the architecture you choose. Inspection tools show real tensors and timings. Export tools produce runnable model code, experiment archives, and editable figures.

1.2 Feature map

Workspace	Main activity	Result
Architecture Studio	Build and edit CNNs, residual models, Transformers, and ViTs	A validated model graph and PyTorch code
Search Space mode	Define variables, conditions, and constraints	A searchable model template
Search Lab	Generate and rank candidates	Candidate architectures, proxy results, and hardware measurements
Dataset Studio	Import and inspect classification data	Versioned data and saved splits
Training Studio	Run CPU or CUDA training	Metrics, checkpoints, and trained weights
Tensor & Performance	Capture tensors and profile execution	Feature maps, gradients, distributions, and timing records
Module Library	Package and reuse subgraphs	Versioned blocks with exposed parameters
Architecture Copilot	Describe changes and review plans	Proposed graph edits, search settings, and tasks
Model Ecosystem	Import models through provider adapters	Model assets and weight snapshots
Figure Studio / Neural CAD	Create publication diagrams and 3D scenes	Editable scenes, SVG, PNG, and PDF

2. Installation and first launch

2.1 Start from the repository

On Windows, install Python 3.10 or later. Clone the repository, enter the project folder, and run the startup script:

```
git clone https://github.com/Panda-China666/NeuroForge-AI.git
cd NeuroForge-AI
.\start.bat
```

The script prepares a local environment on first launch and starts the application at http://127.0.0.1:8787/. Keep the server terminal open while working in the browser.

For a manually managed environment:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e .
python run.py
```

On Windows PowerShell, activate that environment with .\.venv\Scripts\Activate.ps1. Use the environment's Python for the remaining commands.

2.2 Enable CUDA and model adapters

On Windows with a compatible NVIDIA driver, run:

```
.\setup-cuda.bat
.\setup-ecosystem.bat
.\start.bat
```

The Runtime page reports the PyTorch version, CUDA availability, GPU names, and available device IDs. Select auto, cpu, or a detected cuda:N when submitting a task.

2.3 Windows portable application

The desktop build script creates a folder containing NeuroForge.exe, _bootstrap, _internal, and documentation. Extract the complete folder, then double-click NeuroForge.exe. The launcher starts the backend and opens the browser workspace.

Keep the executable and its dependency folders together. A short writable location such as D:\Apps\NeuroForge works well. User datasets and experiments live in the selected data folder.

2.4 Choose a port and data folder

For source execution:

```
$env:NEUROFORGE_DATA = 'D:\NeuroForgeData\vision'
python run.py --port 8877
```

For a desktop build:

```
.\NeuroForge.exe --port 8877 --data-dir 'D:\NeuroForgeData\vision'
```

Open the matching browser address, http://127.0.0.1:8877/. Use a separate data folder for each independently running workspace.

3. Desktop launcher and session control

3.1 Launcher buttons

Button	Action
Start backend	Start or reconnect to the local service
Open workspace	Open the active workspace in your browser
Stop backend	Stop an idle backend managed by the launcher
Data folder	Open the current workspace folder
Choose data folder	Select an existing or new workspace while the backend is stopped
User manual	Open the offline HTML guide

The status area shows the local address, device, PyTorch version, Python version, and selected data folder. Read that location when switching between source and desktop sessions.

3.2 Finish a session

Save the project, then pause or finish active tasks. In a desktop session, choose Stop backend to close the service. Closing the launcher offers three choices: stop the backend, leave it running, or return to the launcher.

Leaving the backend running keeps tasks and the browser workspace available. Reopen the launcher to reconnect. For a source session, finish or pause tasks and use Ctrl+C in the server terminal.

3.3 Logs

Desktop server output is saved under logs/server.log in the selected data folder. Each compute task also retains its own log and error record. Match the task ID and timestamp when investigating an issue.

4. Projects, data folders, and storage

4.1 Workspace contents

Source execution uses the project runtime folder by default. Desktop execution defaults to %LOCALAPPDATA%\NeuroForgeAI\data. An explicitly selected data folder takes precedence.

Item	Purpose
neuroforge.sqlite3	Projects, versions, datasets, jobs, module metadata, and saved documents
datasets/	Dataset arrays, manifests, and split information
artifacts/	Task outputs, checkpoints, captures, and exported assets
Model / operator asset folders	Imported weights, metadata, and registered operator source
logs/	Desktop backend output

Store full workspace backups separately from the program directory. Choose readable project names, save milestones, and export important training runs as ZIP archives.

4.2 Reuse an existing workspace

Stop the service using the data folder, make a backup, then select the original runtime folder in the desktop launcher. Start the backend and check projects, dataset previews, and previous runs.

When moving to another drive, copy the entire data folder. Start from that copy and verify a saved project and checkpoint before continuing work.

4.3 Daily project routine

Save after architecture edits. Keep a named version before search, instantiate promising candidates as new versions, and retain the corresponding training records. Export a completed run and the final figures before archiving a project.

5. Navigate the application

5.1 Sidebar pages

The sidebar groups the tools around one selected project. Architecture Studio is the design entry point. Dataset Studio manages the input. Search Lab and Training Studio run experiments. Module Library, Model Ecosystem, and Custom Operators extend the component set. Tensor & Performance, Optimization Studio, Export Studio, and Neural CAD help analyze and deliver results.

The project switch below the logo changes the active project. Hover compact navigation icons to read their labels. The top bar shows the page and saved version.

5.2 Core objects

Object	Meaning
Project	A saved model workspace with dataset and pipeline references
Model IR	Operators, connections, tensor contracts, and module definitions
Search Space IR	Variables, candidate values, conditions, and constraints
Candidate	A concrete model generated during a search
Job	A submitted proxy, search, training, debug, profile, or optimization task
Module	A reusable subgraph with named ports and exposed parameters
Asset	A versioned weight, operator, or model resource
Figure / Scene	A saved graphical document or 3D arrangement

6. First tutorial: train a CNN

6.1 Create the model

1. Open Architecture Studio and create or select a project.
2. Choose a small CNN or residual template from Template.
3. In Dataset Studio, select Shapes-300 and click Apply to model.
4. Return to the graph. Check the image input, final class count, and Graph ready status.
5. Click Fit canvas and inspect a block in the right-hand Inspector.
6. Save the project with a recognizable name, such as My first CNN.

Shapes-300 is included as a compact classification dataset. It gives you a quick path through the complete workflow before importing your own images.

6.2 Run a proxy analysis

Use the Run menu to start Proxy Analysis. Select auto or a detected CUDA device and begin with SNIP and GradNorm. Inspect the resulting task status, device, elapsed time, and scores. Keep the seed fixed when comparing model variants.

6.3 Start a short training run

Open Training Studio and select New training. Set 3 epochs, batch size 16, learning rate 0.001, and seed 42. Select your device and start the task. Watch the loss and accuracy curves, Worker logs, and checkpoint progress.

A completed run includes epoch metrics, validation results, a saved checkpoint, and final test results for the selected best-validation weights. Export the model archive and save an architecture diagram.

6.4 Extend the exercise

Duplicate the project version, change a block width, and repeat the short run. Compare the two records in Experiment Lab. Then define a small width search in Search Space mode and let Search Lab generate the alternatives.

7. Dataset Studio

7.1 Image ZIP layout

Create one directory per class, then ZIP the class directories:

```
my-images.zip
  cats/
    image-001.jpg
    image-002.png
  dogs/
    image-003.jpg
    image-004.png
```

Class folder names become labels. Prepare enough independent samples per class to support training, validation, and test splits. Supported image formats are decoded and converted through the import pipeline.

7.2 Import and apply images

1. Open Dataset Studio and choose the image ZIP import action.
2. Select the archive, input size, split seed, and dataset name.
3. Run the import and inspect the sample preview, class distribution, and quality report.
4. Review duplicates, decoding issues, and class counts.
5. Save the dataset version and click Apply to model.
6. Confirm the model input dimensions and classification head.

The importer supports square sizes 16, 32, 64, and 128. Use the selected dimensions consistently when configuring the graph and preprocessing steps. Dataset metadata records the source, labels, seed, and split indices.

7.3 Token datasets

Import equal-length integer token sequences through Import token JSON. Paste the records array and set Vocabulary size:

```
[
  {"tokens": [1, 2, 3, 4], "label": "a"},
  {"tokens": [2, 3, 4, 5], "label": "a"},
  {"tokens": [3, 4, 5, 6], "label": "a"},
  {"tokens": [10, 11, 12, 13], "label": "b"},
  {"tokens": [11, 12, 13, 14], "label": "b"},
  {"tokens": [12, 13, 14, 15], "label": "b"}
]
```

Use Vocabulary size 32 for this example. Token IDs start at zero and remain below that value. Supported datasets contain 6–10,000 samples, 2–100 classes, and sequence lengths 1–512. Each class needs at least three independent sequence groups.

Apply the dataset, then choose Use Transformer template. Check sequence length, vocabulary size, and classifier output. The examples folder contains a ready-to-paste token records file.

8. Architecture Studio

8.1 The editing surface

The component library supplies vision, Transformer, embedding, and basic operators. The central canvas shows connections and tensor flow. Select a node to open its parameters and tensor shapes in the Inspector. The console reports structural checks and compilation results.

Add operators from the library, connect their ports, and edit compatible parameters. Use Fit canvas, zoom controls, and Auto layout to navigate. Shift-selection supports working with several nodes at once. Save a version after a meaningful change.

8.2 Parameter workflow

Change one region at a time. Apply changes, inspect its output shape, then verify downstream layers. Convolution output channels affect later inputs. Pooling changes spatial dimensions. Flatten converts the feature dimensions before a Linear layer. Residual Add inputs use matching shapes.

Operator	Useful fields	Check
Conv / Conv Block	out_channels, kernel_size, stride, padding	Spatial shape and downstream channels
Pooling	kernel_size, stride	Feature resolution
Linear	out_features	Class count or feature dimension
Attention	num_heads, embed_dim, dropout	Divisibility and sequence shape
Transformer Stack	depth, num_heads, ffn_ratio	Model scale and residual shape
Module	Exposed parameter bindings	Internal and external tensor contracts

8.3 Save and compare versions

Save a clean baseline before restructuring. Use Architecture Diff to compare versions and candidate graphs. Inspect generated PyTorch through the PyTorch tab. Import architecture JSON when restoring a structural snapshot.

9. Visual Search Space Designer

9.1 Turn a model into a search space

Switch from Design to Search Space. Select the region you want to explore and mark nodes Searchable. Keep the rest Fixed. Conditional nodes and variables become active when their dependency rules match.

For a CNN width experiment, select a block and add out_channels values 16, 24, and 32. Add a second variable for a later block or a repeat count. Save the search space and open Space Summary to inspect the combinations.

9.2 Domain fields

Field	What to enter	Example
Node	The operator or module path	block1
Target	A configuration property or structural choice	config.out_channels
Kind	Discrete or numeric range	discrete
Candidate values	A list of scalar choices	16, 24, 32
Numeric range	Minimum, maximum, and step	4 through 12, step 1
Sampling	Linear or positive log sampling	Linear integer depth
Condition	Dependency variable, relation, value	Enable when branch=true
Default	The reference value	16

Structural variables include operator choice, repeat count, module choice, and enabled branches. Repeated blocks should maintain the necessary input/output shapes. Local variables can target exposed module parameters or internal node paths.

9.3 Constraints and summary

Set parameter, MACs/FLOPs, layer, static memory, and shape constraints for the graph. Add local relations such as divisibility or equality where needed. UI fields marked M use millions; the corresponding API values use absolute integers.

Space Summary lists variable counts, theoretical combinations, searchable nodes, and static legality estimates. The search task then applies actual candidate validation and, when enabled, target-device measurement.

10. Search Lab

10.1 Search strategies

Strategy	Operation	A practical use
Random Search	Samples valid choices	Establish an initial baseline
Evolutionary Search	Mutates and combines candidate configurations	Explore around promising structures
Bayesian GP-UCB	Fits a Gaussian-process surrogate and selects candidates with an acquisition rule	Allocate a finite candidate budget adaptively
Enumerate finite space	Visits the distinct combinations	Inspect a small discrete search space

Choose the project, search space, device, candidate count, seed, and time budget. Start with a small budget, inspect the results, and increase it for the next exploration.

10.2 Hardware-aware search

Enable hardware measurement to record latency, throughput, and peak allocator memory. Use the same device and batch configuration across candidates. Set maximum latency or memory and minimum throughput to filter models against your deployment goals.

Warmup and repeated timing stabilize the measurement protocol. Candidate records retain the actual device and measurement settings so you can compare runs.

10.3 Read and use candidates

The Pareto Explorer plots proxy ranking against model size. Click a point or table row to inspect the corresponding model, raw scores, normalized rank, resource values, and hardware measurements.

Select two candidates for Architecture Diff. Use Compare with template to inspect the search changes. Choose Instantiate as new version to create a fixed model, save a candidate snapshot, or start training from the selected candidate. Retain the search record to preserve its configuration and lineage.

11. Proxy analysis

11.1 Available signals

Proxy analysis evaluates initialized architectures with a small computation budget. Use the resulting scores and rankings to select candidates for training. Keep the dataset, seed, device, and sample protocol consistent within a comparison.

Proxy	Signal	Reading the record
SNIP	Weight and supervised-gradient sensitivity	Score, selected samples, and gradient computation
SynFlow	Positive-weight signal flow	Initialization and floating-point execution details
NASWOT	Agreement among ReLU activation patterns	Activation kernel and validity status
GradNorm	Aggregate layer-gradient magnitude	Gradient norms under the selected data batch
ZiCo	Gradient variation across fixed microbatches	Gradient statistics and valid layer count
Fisher	Activation-gradient sensitivity	Channel sensitivity statistics
Jacobian Covariance	Spectrum of input-Jacobian correlations	Input-gradient correlation measurements

Params, MACs, and FLOPs describe architecture scale. Each proxy has its own scale; normalized rankings help combine their observations. Validation and test accuracy are available in the corresponding training and evaluation records.

11.2 Compare architectures

Save the baseline architecture, select the proxy methods, choose a device, and run the analysis. Inspect each method's status, raw score, elapsed time, and peak memory. Use repeated seeds when studying ranking stability.

Select a diverse group of promising candidates for short training. Keep both the proxy record and the training outcome to compare how the signals behave on your task.

12. Training Studio

12.1 Training configuration

Setting	Accepted configuration	How to use it
Device	auto, cpu, detected cuda:N	Choose the execution device
Epochs	1–200	Use a short run first, then increase the budget
Batch size	2–128	Balance update size and available memory
Learning rate	0.000001–0.1	Set the AdamW learning rate
Weight decay	0–1	Set optimizer regularization
Seed	Integer from 0 through 2^31−1	Keep it fixed for controlled comparisons
AMP	CUDA mixed precision	Reduce activation precision during training
Gradient accumulation	1–32 steps	Accumulate several microbatches per update
Auto batch	Enabled or disabled	Probe a fitting microbatch after allocation pressure
Early stopping patience	0–50	Zero keeps the full epoch schedule
Early stopping delta	0–1	Set the required validation improvement

The runtime uses AdamW and a cosine learning-rate schedule. Review the saved configuration alongside the epoch metrics and actual effective batch settings.

12.2 AMP and accumulation

For a memory-efficient run, select CUDA, enable AMP, and start with batch size 8 and accumulation 4. This combines four microbatches for each optimizer update. The run records effective precision, microbatch size, and accumulation.

BatchNorm observes the microbatch used by each forward pass. When comparing runs, retain both the effective optimizer batch and the physical microbatch configuration in your notes.

12.3 Monitor the run

The status area shows the device name, epoch and step progress, timing, throughput, checkpoint, and memory counters. Loss and accuracy charts track training and validation separately. Worker logs record initialization, execution settings, epoch summaries, and checkpoint saves.

Training throughput uses the recorded training protocol. Profiler inference throughput uses its own batch and timing protocol. Read each measurement with its configuration when comparing performance.

13. Task states, pause, resume, and checkpoints

13.1 State flow

```
queued → running → completed
          ↓
        pausing → paused → queued
          ↓
       cancelling → cancelled
          ↓
        failed
```

Queued tasks wait for the local compute worker. A running task is executing. Pausing and cancelling indicate a pending control request. The task then reaches its saved paused state or final cancelled state.

13.2 Pause and resume training

1. Open the active training run and select Pause.
2. Wait for the current epoch's safe checkpoint point and the Paused status.
3. Check the latest checkpoint epoch and saved metrics.
4. Keep the referenced dataset, model assets, and task files together.
5. Choose Resume to continue from the saved optimizer, scheduler, random state, and epoch.

Use Run again when starting a fresh task with the recorded setup. Use the task logs to inspect any recovery message after an interrupted session.

13.3 Checkpoint files

File	Contents	Typical use
checkpoint.pt	Latest epoch, model, optimizer, scheduler, and random states	Resume training
best.pt	Weights selected by best validation performance	Final evaluation and inference
model.py / architecture.json	Model implementation and structural configuration	Rebuild the matching architecture
Experiment metadata	Dataset, split, pipeline, settings, environment, and metrics	Reproduce and compare the run

Export the completed run archive to keep its model and configuration together.

14. Visual Tensor Debugger and profiler

14.1 Capture tensors

Open Tensor & Performance and choose Capture tensors. Select the current initialized model or a matching completed training checkpoint. Set the device and inspection batch, then submit the task.

After completion, select a node from the left-hand list. Read the tensor shape, dtype, mean, standard deviation, min/max, zero fraction, and finite-value statistics. Switch between Activations and Gradients and inspect available parameter-gradient rows.

14.2 Explore feature maps

Choose a feature channel to display its heatmap. The preview uses the first sample and adapts large spatial maps for display. The raw-values panel shows a numeric excerpt. Download tensor NPZ files for saved arrays within the capture budget.

Capture a smaller batch when exploring a large model. Keep the capture task ID with the checkpoint and project version to identify exactly which model produced the features.

14.3 Profile execution

Choose Run profiler, select the device and checkpoint, and set the inspection batch and benchmark repeats. The runtime records node timings using CPU clocks or CUDA events, and benchmarks the whole model after warmup.

Read latency, throughput, output bytes, and allocator measurements together. Use Graph overlay to annotate the matching model graph. Rerun profiling after structural edits to update the timing view.

15. Module Library and reusable subgraphs

15.1 Create a reusable block

Select connected nodes in Architecture Studio with Shift-selection or the selection box. Open the context menu and choose Create Module / Save as Reusable Block. Enter a name, description, and category, then review the input and output ports.

The packaged node replaces the selected region and carries its internal graph. Save it to Module Library to insert it into another project with compatible input and output shapes.

15.2 Expose parameters

Map an external module property to an internal configuration field, such as channels to a convolution's out_channels. The Inspector then edits the module through that public parameter. Search Space can target the same exposed property.

A repeat count or module-choice variable can explore different block configurations. Validate the surrounding tensor shapes after each packaging or binding change.

15.3 Navigate and version modules

Enter a module to edit its internal graph. Use the breadcrumb to return to the parent model. Module Library records the name, category, version, ports, parameter schema, source, and compatibility metadata.

Keep a version before changing a widely reused block. Save the revised block as a new module version, then select it in the projects where you want to use it.

16. Transformer and Vision Transformer workflows

16.1 Build a native ViT

Start with a ViT template. The graph combines Patch Embedding, an optional Class Token, positional embeddings, a Transformer Stack, LayerNorm, token pooling, and a Linear classifier.

For a 32×32 image and patch size 4, the model creates 8×8 patches. A class token adds one position. With embedding dimension 64, the sequence shape is 65×64 when the class token is enabled.

The shape checker verifies patch counts, hidden dimensions, attention-head divisibility, residual compatibility, and classifier output. Choose a head count that divides the embedding dimension.

16.2 Search the encoder

Keep Patch Embedding and the classifier Fixed. Mark the encoder Searchable and define depth values 4 through 12 and head choices 4, 8, and 16. With a compatible hidden dimension, this defines 27 combinations before additional constraints.

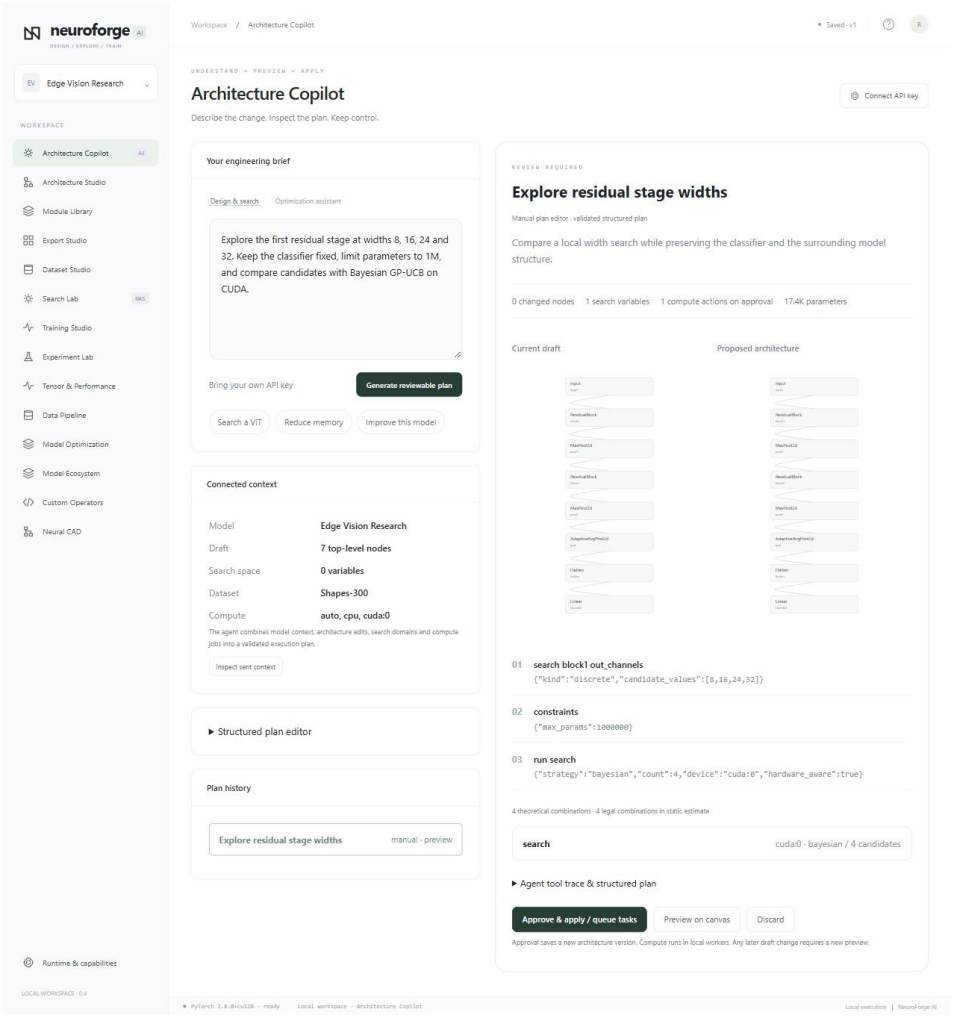
Set your parameter and hardware budgets, save the space, then run Bayesian GP-UCB or finite enumeration. Inspect the resulting candidates and instantiate a selected architecture before training.

16.3 Token Transformer classification

Apply a token dataset, then choose the token Transformer template. Confirm vocabulary size and sequence length. Embedding produces the hidden sequence; positional information and Transformer blocks process it; pooling and the classifier produce class logits.

Use the same data contract through proxy analysis, debugging, search, and training. Retain the token dataset manifest and model configuration in the experiment record.

17. AI Architecture Copilot



Architecture Copilot plan preview

17.1 Connect a provider

Open Architecture Copilot and choose Connect API key. Enter the API base URL, model ID, your API key, and the protocol used by the service. The application supports Responses API and compatible Chat Completions services with tool calling and structured responses.

For an OpenAI service, the base URL is https://api.openai.com/v1; enter a model ID available to your account. A local compatible service may use a localhost HTTP base URL. Save the settings, then return to your engineering brief.

17.2 Describe the goal

Give the agent the region, editable variables, constraints, search strategy, device, and desired tasks. For example:

Keep the current ViT patch embedding and classifier fixed. Search Encoder depth from 4 through 12 and attention heads 4, 8, and 16. Set a 50M parameter budget, use Bayesian GP-UCB, and measure candidate latency on cuda:0.

Copilot reads the model IR, current search space, dataset metadata, available devices, and relevant measurements. Inspect sent context shows the engineering context used for the request.

17.3 Review and apply a plan

The preview displays graph changes, searchable variables, constraints, task settings, and the proposed architecture. Open the canvas preview to inspect the affected region. Click Approve & apply to save the model changes and queue the reviewed tasks.

Editing the project after a preview calls for a fresh plan using the updated context. Plan history preserves the proposal and its execution status. The Structured plan editor provides a direct way to enter and validate a plan JSON document.

17.4 Provider sessions

The API key stays in backend session memory for up to eight hours. Disconnect the provider or stop the backend to clear the session. Keep configuration and credentials in their dedicated UI fields; export project and experiment files through the normal workspace actions.

18. Architecture Optimization Assistant

18.1 Plan an optimization

Switch Copilot to Optimization assistant and state a measurable goal, such as reducing peak memory or shortening inference latency. Give the target device, evaluation dataset, and acceptable quality trade-off.

Use a three-stage workflow. First, measure and save the baseline. Second, preview a controlled configuration or structural change. Third, train or evaluate the variant and compare the matching records.

18.2 Compare real results

Verify observed optimization lets you select a Baseline run and an Optimized run. The report aligns the dataset, pipeline, seed, epochs, device, environment, and effective batch settings, then shows memory and accuracy changes.

For example, a change from 75.84 MiB to 68.13 MiB corresponds to a 10.16% reduction. Use the reported values to guide another iteration. Keep the baseline and variant run IDs with the final model.

18.3 Useful changes to explore

AMP, microbatch size, and gradient accumulation adjust the training runtime. Local width or depth searches explore architecture alternatives. Bottleneck replacements can change selected imported image stages. Optimization Studio supplies pruning, dynamic Linear INT8, and ONNX jobs for the delivery workflow.

19. Model Ecosystem

19.1 Provider catalog

Provider	Model families	Workflow
torchvision	ResNet-18/34/50, EfficientNet-B0, ConvNeXt-Tiny, ViT-B/16, Swin-T	Import a model snapshot and inspect its boundaries
timm	ResNet-18, EfficientNet-B0, ConvNeXt-Tiny, ViT-Tiny, DeiT-Tiny, Swin-Tiny	Import pooled-feature backbones and task heads
Hugging Face	BERT, GPT-2 classification, ViT, Swin	Import configuration and classification weights through the adapter

Install the ecosystem dependencies, select the dataset, and choose a provider and model. Review input size and class count before importing. Download options control configuration and pretrained-weight retrieval.

19.2 Import a model

1. Open Model Ecosystem and select the provider and model ID.
2. Choose initialized weights or enable Pretrained with the download option.
3. Start the import job and follow its download, construction, and probing logs.
4. Inspect the weight status, module tree, tensor contracts, and classifier notes.
5. Apply the graph preview to the project and save a version.
6. Run a small profile or training task to check the full data-to-model path.

Imported backbones retain their recorded weight snapshots. The task head is configured for your dataset. Inspect the import notes to see which classifier parameters were initialized. Provider input contracts, including the 224×224 torchvision pretrained ViT setting, guide model selection and preparation.

19.3 Search a replacement region

Select an image feature-map boundary and choose Preview searchable replacement. A native bottleneck module is created with a checked output contract. Review it, apply it, and define candidate widths for the exposed parameter.

Search that local region while keeping the rest of the imported model fixed. Use measured device latency, throughput, and memory to select a candidate, then train and export the revised model.

19.4 Asset records

Model assets save provider information, configuration, input/output contracts, weight status, hashes, and local snapshots. Related model and experiment exports include the assets and adapter code used for reconstruction. Use hardware measurements when comparing provider blocks.

20. Code → Visual and Architecture Diff

20.1 Import a model definition

Open the Code → Visual tool, paste a static nn.Module or nn.Sequential definition, and specify input type, shape, and class name. The parser maps layer declarations and forward data flow to an editable graph.

The import handles supported layer constructors, intermediate variables, module calls, additions, and batch-preserving Flatten operations. For a complete model-and-weight import, use the Model Ecosystem workflow in chapter 19.

20.2 A ready-to-use CNN

```
from torch import nn

class TinyClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(3, 8, 3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.flat = nn.Flatten()
        self.head = nn.Linear(8, 3)

    def forward(self, x):
        x = self.relu(self.conv(x))
        x = self.pool(x)
        x = self.flat(x)
        return self.head(x)
```

Set the input shape to [3,32,32] and class name to TinyClassifier. Preview the generated graph, inspect the output [3], apply it, and save. The examples folder includes this model as a file.

20.3 Compare architecture versions

Select left and right models in Architecture Diff. Review added, removed, or changed nodes, parameters, and edges. Stable node IDs help trace consecutive versions. Compare a candidate with its template to inspect the exact structural changes selected during search.

After reviewing a change, run graph validation and a small execution task, then continue with training or export.

21. Custom Operator SDK

21.1 Write an operator

Define class Operator(nn.Module) with constructor parameters and a differentiable forward pass. This example scales each channel using a learned gate:

```
import torch
from torch import nn

class Operator(nn.Module):
    def __init__(self, channels=8):
        super().__init__()
        self.gain = nn.Parameter(torch.ones(1, channels, 1, 1))

    def forward(self, x):
        return x * torch.sigmoid(self.gain)
```

Use constructor JSON {"channels":8} and probe shape [8,16,16]. The probe shape lists feature dimensions after the batch dimension.

21.2 Register and reuse it

Enter a name, description, source, parameters, and probe shape in Custom Operators. Review the code and acknowledge trusted execution. The worker constructs the module, runs finite-value and differentiability checks, and records the resulting tensor contract.

A successful registration creates a component-library entry with a source hash and registry version. Insert it where the model shape matches its contract, then run a small debug or profile task. Register a new version after changing the source or constructor settings.

21.3 Registration reference

Source size is up to 50 KB. A probe shape has 1–3 dimensions, each from 1 through 1024, and up to 1,000,000 elements. The output is a finite, differentiable Tensor that retains the batch dimension. The registration probe permits up to 4,000,000 output elements and 50M parameters.

The worker executes reviewed Python code with the current user's permissions. Keep operator source and metadata together with the projects that use them.

22. Experiment Lab and model export

22.1 Read experiment records

Each job stores the submitted model, dataset ID, data pipeline, device, seed, and task configuration. Its outputs include status, logs, measurements, metrics, and artifact references. Use those snapshots to identify the exact setup behind a result.

Compare runs with aligned data splits and training budgets. Read validation and test accuracy alongside model size, training duration, and measured device cost. Save representative candidates and the final selected model version.

22.2 Export choices

Export	Included material	Use
Model source	PyTorch code and architecture configuration	Rebuild and develop the model
Training archive	Model files, checkpoints, configuration, logs, and records	Resume or reproduce an experiment
Referenced assets	Imported weight snapshots and operator adapters	Reconstruct an asset-based model
Figure files	SVG, PNG, PDF, and editable scene documents	Publish or present the architecture

Download the task archive, open it in a separate folder, and check its README and file list. Keep the original data and dataset manifest with your experiment collection. Use chapter 27 for a whole-workspace backup.

22.3 Model optimization

Optimization Studio offers magnitude pruning, CPU Linear dynamic INT8, and ONNX export. Select a completed checkpoint and configuration, run the task, and inspect the output archive and inference checks.

Compare the optimized model's size, latency, and accuracy under a saved evaluation protocol. Use ONNX Runtime results to check the exported inference path and retain the input shape and preprocessing configuration with the artifact.

23. Visual Data Pipeline

23.1 Available image steps

Operator	Default parameters	Default scope	Result
flip	probability=0.5	train	Horizontal flip per selected sample
rotate	quarter_turns=1	train	Rotation in 90-degree increments
brightness	factor=1.2	all	Brightness scaling in the image value range
contrast	factor=1.2	all	Contrast adjustment around the spatial mean
noise	std=0.03	train	Gaussian image noise
cutout	size=8	train	A randomly located zero-valued square
crop	padding=4	train	Padded random crop retaining input dimensions
normalize	mean=0.5, std=0.5	all	Scalar normalization
resize	size=32	all	Bilinear square resize

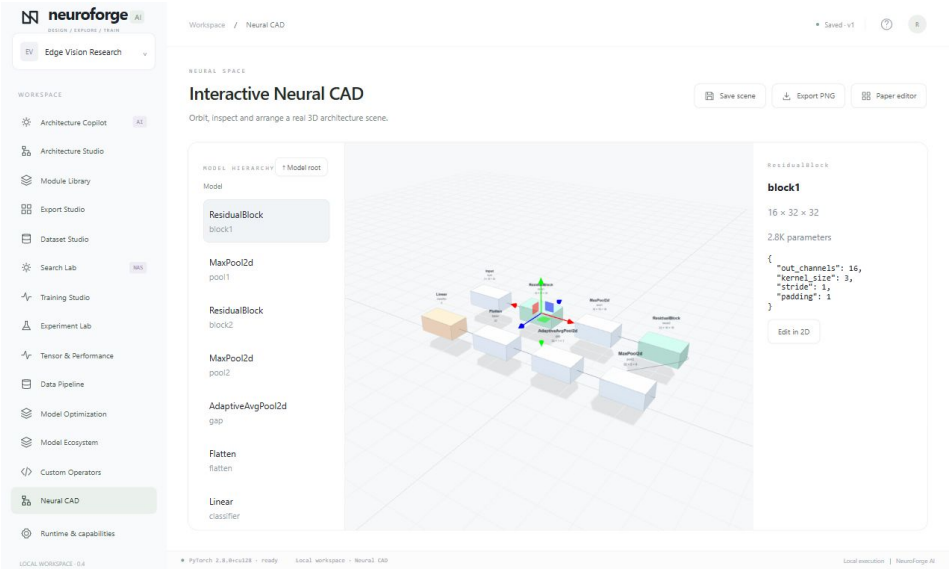
Train-scoped steps apply to the training data path. All-scoped steps also apply during validation and test. Set the scope explicitly when choosing an augmentation policy.

23.2 Build and save a pipeline

Add steps in Data Pipeline, drag to reorder them, and edit the selected step's parameters and scope. Inspect Before / After previews with a fixed seed. Apply the pipeline to the project and submit the next task to use its saved snapshot.

Place brightness, contrast, noise, and similar range-based transforms before normalization. Keep resize dimensions aligned with the model's input H/W. The image pipeline accepts N,C,H,W floating tensors and up to 16 processing steps.

24. Export Studio, Figure Studio, and Neural CAD



Interactive Neural CAD

24.1 Create a paper diagram

Save and validate the architecture, then choose Export 2D. Select a style, orientation, output size, and label options. Include names, tensor shapes, parameters, stage groups, and repeat labels as useful to your audience.

Preview the result and export SVG for vector editing, PDF for document placement, or PNG for image workflows. Group complex subgraphs into modules to produce a readable overview. Generate figures from the final selected architecture version.

24.2 Edit individual elements

Open Figure Studio 2.0 from the generated figure. Select text, cards, connectors, or projected faces. Drag to move, edit properties, add annotations, group items, duplicate elements, and use Undo while arranging the page.

Save the figure document or Scene JSON for continued editing. Export SVG, PNG, or PDF when ready. Keep the figure document with the corresponding project version so labels and architecture remain easy to track.

24.3 Arrange a 3D scene

Open Neural CAD for an interactive WebGL view. Drag to orbit, right-drag to pan, and use the wheel to zoom. Select blocks in the scene or hierarchy. The Inspector displays their shapes and parameters; the translation gizmo positions a selected block.

Enter native modules through the hierarchy and return with the breadcrumb. Save scene to retain the camera and block positions. Export PNG produces a 3200×2000 presentation image. Paper editor opens the editable figure workflow.

24.4 Choose an output for the audience

Use a clean 2D SVG for a methods section, a compact diagram for a README, and a 3D PNG for a presentation or cover. Add a title, readable labels, and a brief legend. Keep text size and stage colors consistent across related figures.

25. Python SDK, CLI, and API

25.1 Connect the CLI

Start the backend, then use the Python environment that contains NeuroForge:

```
python -m neuroforge --help
python -m neuroforge system
python -m neuroforge projects
python -m neuroforge datasets
```

For another port, place the URL before the subcommand:

```
python -m neuroforge --url http://127.0.0.1:8877 system
```

In a desktop build, use _internal/python/python.exe as the interpreter. Copy the actual project and job IDs from the command output. Store configuration files as UTF-8 JSON.

25.2 Submit and control tasks

```
python -m neuroforge proxy --project PROJECT_ID --device auto --wait
python -m neuroforge train --project PROJECT_ID --device auto --wait
python -m neuroforge job JOB_ID
python -m neuroforge control JOB_ID pause
python -m neuroforge control JOB_ID resume
python -m neuroforge export JOB_ID --output training-result.zip
```

Use --config docs/examples/train-config.json to supply training options or the corresponding proxy configuration file. --device selects the requested runtime. --wait returns at a final or paused state; inspect status, error, logs, and result fields.

25.3 Automate with Python

```
from neuroforge.sdk import NeuroForge

client = NeuroForge("http://127.0.0.1:8787")
project = next(p for p in client.projects() if p["name"] == "My first CNN")
job = client.train(project["id"], epochs=3, batch_size=16, device="auto")
finished = client.wait(job["id"], timeout=1800)
print(finished["status"], finished.get("error"))
if finished["status"] == "completed":
    client.export(job["id"], "training-result.zip")
```

Other SDK methods provide code import, architecture diff, search, tensor debug, profiling, model imports, operator registration, plan previews, and optimization reports. Interactive API documentation is available at the backend's /docs page.

25.4 Preview a structured plan

```
python -m neuroforge preview-plan docs/examples/profile-plan.json --project PROJECT_ID
```

Review the returned plan, proposed model, search space, and tasks. The SDK's apply_plan takes that preview ID, matching graph and search space, and confirm=True. This makes plan review part of an automated engineering workflow.

26. Troubleshooting

26.1 Find the cause

Symptom	Check	Action
Browser connection error	Server address and process	Start the backend and open the matching port
Port already in use	Running service and selected workspace	Choose another port, such as 8877
A project appears empty after switching editions	Selected data folder	Select the original runtime folder and reopen the project
Slow first launch	Server log and environment initialization	Follow startup progress, then check the first logged error
CUDA unavailable	Runtime device list and system driver	Choose CPU for the task or prepare the matching CUDA environment
CUDA out of memory	Batch, input, activations, and model scale	Reduce the microbatch, enable suitable AMP/auto batch, and rerun
Device mismatch in a custom operator	Newly created tensors and buffers	Create tensors on the input device or register buffers, then validate a new version
Shape mismatch	First changed layer and downstream contracts	Inspect the graph's shapes and correct the affected dimensions
Attention-head error	Embedding width and head choices	Choose divisible combinations
Fewer search results than requested	Space summary and legal unique combinations	Inspect the completed finite space or extend its variables
All candidates filtered	Structural and hardware constraints	Start with broader budgets, then tighten them progressively
Proxy method reports a failure	Individual proxy status and error	Use the method's recorded input and architecture requirements
Pause is still pending	Current epoch and checkpoint activity	Wait for the epoch's saved safe point
Checkpoint restore error	Matching architecture, dataset, and assets	Restore the saved setup and referenced files
Provider HTTP 401 or 403	API key, model ID, and account access	Correct provider settings and reconnect
Provider HTTP 404	Base URL, route, and selected protocol	Use the service base URL and matching API protocol
Plan expired or changed	Project revision and preview context	Generate and review a fresh plan

Symptom	Check	Action
Model download error	Repository ID, network, and download selection	Check the chosen resource and retry the import
Figure labels refer to an older model	Figure's source version	Generate a new figure from the final architecture
Backend stop is deferred	Active tasks	Finish, pause, or cancel them, then stop the backend

26.2 Record diagnostic information

Collect the application version, task ID, device, input shape, model name, steps, and error text. Save the relevant time range from server and worker logs. A small model IR and configuration file help reproduce an issue.

Keep credentials in provider settings and share only the diagnostic material needed for the issue. Preserve the existing workspace while testing a fix in a separate folder.

27. Backup, migration, upgrades, and removal

27.1 Make a full backup

Finish active tasks or pause training at a saved checkpoint. Stop the backend, then copy the entire data folder to a dated backup location. Include the database, dataset arrays, manifests, task artifacts, imported weights, and operator assets.

Save important exported model ZIPs and original input data alongside the backup. SQLite can use WAL and SHM side files during operation, so copying the stopped workspace as a whole preserves the complete set.

27.2 Restore or upgrade

Open a backup copy as a data folder and start the application. Check the project list, a dataset preview, and a previous training run. For a new application version, keep the older workspace backup, prepare the new code or program folder, and select the intended data directory.

Use one backend at a time for a given data folder. Stop the service before moving its directory or replacing runtime files.

27.3 Remove program files

Stop the backend, then remove the source checkout or portable program folder you want to discard. The selected user data folder and your exported artifacts can be retained separately. Keep the files needed for your backup and future experiments.

28. Three complete project recipes

28.1 Image dataset to a searched classifier

Prepare class directories and import the ZIP. Review labels and splits, then apply the dataset to a small CNN. Save a baseline. Search the last block's width and repeat count with an initial budget of 8–16 candidates. Inspect proxy and hardware results, instantiate a candidate, and run a short training test. Increase the training budget for the selected architecture.

Finish by exporting the model and checkpoint archive, dataset manifest, search record, and final paper figure. Save the search template for the next dataset.

28.2 ResNet import to a local replacement

Import a ResNet, save the original version, and measure a baseline. Preview a searchable replacement at an eligible image stage. Define candidate widths for that module and run hardware-aware search. Instantiate a variant, train it under a matching protocol, and compare structure, accuracy, latency, and memory.

Export the selected model with its referenced asset snapshots and comparison records.

28.3 ViT with Copilot-assisted search

Start from the ViT template and verify image size, patch count, hidden dimension, and classifier. Ask Copilot to search encoder depth 4–12 and heads 4, 8, and 16 with a 50M parameter budget and CUDA measurements. Review the 27-combination local space and proposed tasks, then approve the plan.

Select a candidate, instantiate it, train, inspect its tensors, and create a publication figure. Preserve the plan and search history alongside the trained model.

29. Configuration and terminology reference

29.1 Local configuration reference

Object	Configuration
Top-level model graph	1–256 nodes
Expanded graph	Up to 2048 operators under the compiler budget
Native static analysis	Up to 250M parameters, 32M activation elements, and 10G MACs
Candidate budget	2–256 requested candidates, bounded by legal unique combinations
Search domains	Up to 64 variables and 64 scalar choices per discrete domain
Training	1–200 epochs and batch size 2–128
Inspection	Batch 2–16; benchmark repeats 2–30
Image pipeline	Up to 16 steps
Figure document	Up to 12,000 graphical primitives
Model assets	Adapter configuration and 250M parameter asset budget

Choose resource settings with the input dimensions, optimizer state, activation size, and target device in view. Profile progressively as you expand the model.

29.2 Glossary

Term	Meaning
IR	Structured model or search-space representation
Model graph	Tensor and operator dependencies
Search space	Domains, conditions, and constraints for model generation
Proxy	A low-cost signal used for architecture ranking
Candidate	A generated concrete architecture with recorded measurements
Checkpoint	Saved model and training state
AMP	Automatic mixed precision on the selected CUDA path
Microbatch	Samples used by one forward/backward pass

Term	Meaning
Accumulation	Gradient collection across several microbatches
MACs / FLOPs	Compute-scale metrics under a stated counting convention
Peak memory	The recorded runtime allocator peak for that task
Contract	Input/output shape and type expectations
Asset	Versioned weights, source, or model resources
Scene	A saved graphical or spatial arrangement
BYOK	Connect your own model-service API key
Pareto frontier	Candidates representing trade-offs across objectives

30. Working checklist, files, and further reading

30.1 A complete project

• Save the dataset version, class mapping, split, and preprocessing pipeline.

• Validate the architecture and save its baseline version.

• Save the search space and inspect candidate configurations and measurements.

• Instantiate the selected candidate and retain the search record.

• Finish training with metrics, best weights, and a resume checkpoint.

• Capture representative tensors and profile the final model.

• Export the model archive, editable figure, and chosen presentation formats.

• Back up the complete workspace.

30.2 Documentation files

This manual is available as Markdown, PDF, and offline HTML. The examples directory contains token records, proxy and training configurations, a CNN definition for Code → Visual, a custom operator, and a structured profiling plan.

The repository README introduces the product and links to the English and Chinese manuals. The desktop build guide explains how to create the Windows application directory.

30.3 Official references

• PyTorch documentation (https://pytorch.org/docs/stable/index.html)

• torchvision models (https://docs.pytorch.org/vision/stable/models.html)

• timm documentation (https://huggingface.co/docs/timm/index)

• Transformers documentation (https://huggingface.co/docs/transformers/index)

• PyInstaller operating modes (https://pyinstaller.org/en/stable/operating-mode.html)

Start with a small end-to-end project, then reuse its data preparation, architecture template, search configuration, and export workflow for the next experiment.